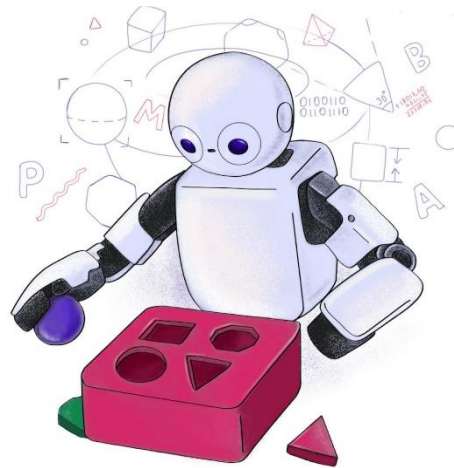


TP558 - Tópicos avançados em Machine Learning:

NAS-Bench-101: Towards Reproducible Neural Architecture Search

Proceedings of the 36th International Conference on Machine Learning, ICML 2019

Inatel



Matheus Ferreira Silva
matheus.f@inatel.br

Contexto do Tema

- **NAS, Neural Architecture Search:**
- A área que busca **descobrir automaticamente arquiteturas neurais**, reduzindo a dependência de tentativa e erro.



Algorithm 1 Single-objective NAS algorithm.

Require: S008 data $\mathcal{D}_{008}$, S009 data $\mathcal{D}_{009}$, trials T , max epochs E

Ensure: best configuration Θ^* and architecture $\mathcal{A}^*$

- 1: split $\mathcal{D}_{008}$ into $\mathcal{D}_{\text{train}}$ (80%) and $\mathcal{D}_{\text{val}}$ (20%)
 - 2: define family set $\mathcal{F}$ with candidate families:
CNN 1D, CNN 2D, Transformer-based, GNN, and CNN 1D+GNN
 - 3: initialize an Optuna study with TPE and Hyperband
 - 4: **for** trial = 1 to T **do**
 - 5: **Preprocess inputs:**
 fit a standard scaler on the training-set GPS data, then transform the training and validation GPS data
 one-hot encode LiDAR (20, 200, 10) $\rightarrow$ (20, 200, 4) and reshape to (4000, 4)
 tile GPS (2,) $\rightarrow$ (4000, 2) and concatenate to obtain (4000, 6)
 - 6: Sample family $f \in \mathcal{F}$
 - 7: sample family-specific hyperparameters from f
 - 8: **Sample optimization hyperparameters:**
 $\theta_{\text{opt}} \in \{\text{SGD, Adam, AdamW, Lion, RMSprop}\}$
 $\theta_{\text{lr}} \sim \log \mathcal{U}(10^{-4}, 10^{-2})$, $\theta_{\text{bs}} \in \{32, 64, 128, 256\}$
 - 9: build model $\mathcal{A}$ from f with the sampled hyperparameters
 - 10: compile $\mathcal{A}$ with sparse categorical cross-entropy and $(\theta_{\text{opt}}, \theta_{\text{lr}})$
 - 11: train on $\mathcal{D}_{\text{train}}$ with batch size θ_{bs} for up to E epochs with early stopping
 - 12: report validation sparse categorical cross-entropy loss on $\mathcal{D}_{\text{val}}$ each epoch and prune if unpromising
 - 13: store final validation loss $\mathcal{L}_{\text{val}}$
 - 14: **end for**
 - 15: select $(\Theta^*, \mathcal{A}^*) = \arg \min_{\Theta} \mathcal{L}_{\text{val}}$ over completed trials
 - 16: retrain $\mathcal{A}^*$ with Θ^* under the S008 protocol and evaluate once on $\mathcal{D}_{009}$
 - 17: **return** Θ^* and $\mathcal{A}^*$
-

Contexto do Tema

- A motivação vem do fato de que arquiteturas projetadas por especialistas podem ter alto desempenho, mas exigem **muito ajuste empírico e conhecimento especializado**.
- **Neural Architecture Search, NAS**, busca automatizar o projeto de arquiteturas neurais, substituindo parte do ajuste manual de operações, conexões e larguras por um processo de busca orientado por desempenho e custo.

Contexto do Tema

Antes do NAS-Bench-101, avaliar NAS normalmente exigia:

- escolher uma arquitetura candidata
- treinar essa arquitetura do zero
- avaliar validação e teste
- repetir o processo para muitas arquiteturas
- repetir execuções para lidar com variação aleatória

O gargalo não era apenas encontrar boas arquiteturas, mas avaliar cada arquitetura de forma confiável.

Contexto do Tema

O NAS-Bench-101 transforma a avaliação de arquiteturas em uma consulta a uma base tabular. Em vez de treinar uma rede candidata durante cada experimento de NAS, o método permite consultar:

- acurácia de treino
- acurácia de validação
- acurácia de teste
- tempo de treinamento
- número de parâmetros

Assim, o custo principal de avaliação é substituído por uma busca em uma tabela pré-computada.

Introdução do artigo

Os autores construíram um benchmark com:

- search space compacto, mas expressivo
- arquiteturas CNN baseadas em células
- 423 mil arquiteturas únicas após remoção de duplicatas
- treinamento e avaliação em CIFAR-10
- mais de 5 milhões de modelos treinados
- uso de mais de 100 TPU years de computação

O resultado é um dataset público que mapeia arquiteturas para métricas de treinamento e desempenho.

Introdução do artigo

O artigo contribui com três elementos centrais:

- Um dataset público e padronizado para pesquisa em NAS.
- Uma análise global do search space, permitindo estudar propriedades das arquiteturas.
- Um protocolo rápido para comparar algoritmos de busca, como random search, evolução e Bayesian optimization.

O artigo não está dizendo: “criamos a melhor CNN para CIFAR-10”.

Ele está dizendo: “criamos um benchmark controlado para estudar algoritmos de NAS”.

Trabalhos relacionados

Arquiteturas projetadas manualmente

A literatura inclui abordagens para otimização de redes neurais de forma manual.

Redes como Inception, ResNet e DenseNet mostram que a organização das camadas tem impacto direto no desempenho.

NAS com alto custo computacional

Os trabalhos anteriores mostram métodos baseados em reinforcement learning e evolução mostraram bons resultados.

O problema é que muitos desses métodos ainda exigem alto custo de busca.

NAS mais eficiente

Métodos com weight sharing, relaxação diferenciável e avaliação parcial reduzem custo, mas dificultam comparação justa.

A lacuna principal é a falta de um ambiente padronizado para comparar algoritmos de busca.

Modelo de Sistema

O NAS-Bench-101 é modelado como um benchmark tabular de arquiteturas neurais.

A proposta dos autores é substituir o ciclo tradicional:

propor arquitetura → treinar do zero → validar → comparar

por uma consulta direta:

propor arquitetura → consultar métricas pré-computadas → comparar

O sistema é composto por quatro blocos:

- search space de células CNN
- codificação das arquiteturas
- pipeline padronizado de treinamento
- protocolo de consulta para benchmarking de NAS

Modelo de Sistema

Todas as arquiteturas seguem a mesma macroestrutura:

Stem $\rightarrow$ [Cell $\times$ 3] $\rightarrow$ Downsample $\rightarrow$ [Cell $\times$ 3] $\rightarrow$ Downsample $\rightarrow$ [Cell $\times$ 3] $\rightarrow$ GAP $\rightarrow$ Dense

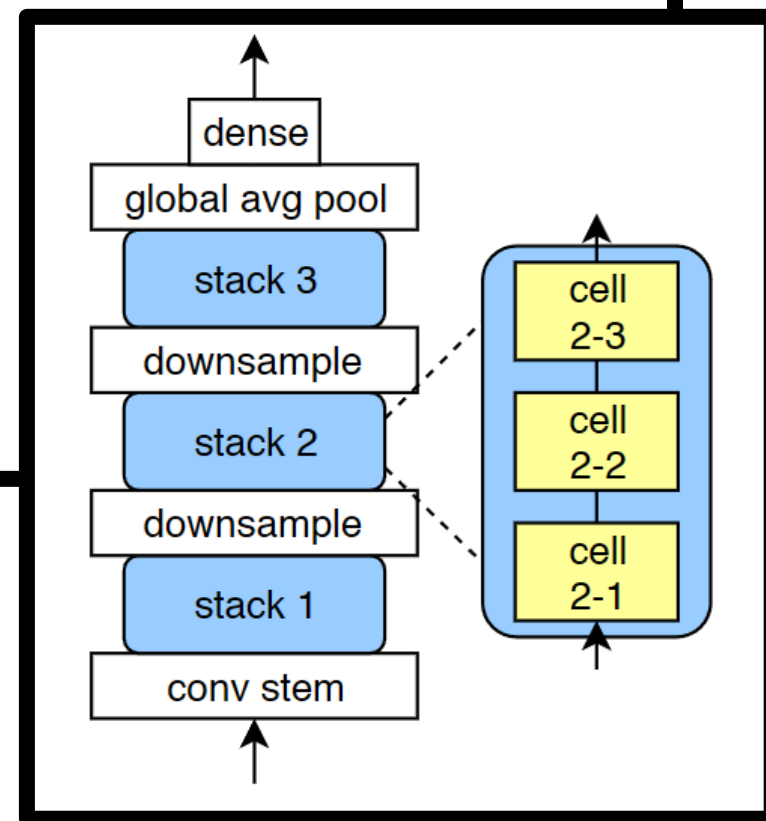
Onde:

- **Stem**: convolução inicial 3×3
- **Cell**: bloco neural buscado pelo NAS
- **Downsample**: reduz altura e largura, dobra canais
- **GAP**: global average pooling
- **Dense**: classificador final

A busca altera apenas a célula.

Vantagens dessa escolha:

- reduz o tamanho do search space
- mantém o treinamento padronizado
- permite comparar arquiteturas sob a mesma rede externa
- segue o padrão usado em muitos trabalhos de NAS para classificação
- concentra a análise nas operações e conexões internas da célula



Modelo de Sistema

Cada célula é um **DAG**, um grafo direcionado acíclico. Existem dois nós especiais:

- **IN**, entrada da célula.
- **OUT**, saída da célula.

- v_{in} : tensor de entrada da célula
- v_{out} : tensor de saída da célula
- $v_1, \dots, v_5$: nós intermediários
- E : conexões entre nós
- $\mathbf{o}$: operações dos nós intermediários

Cada célula é um grafo acíclico direcionado:

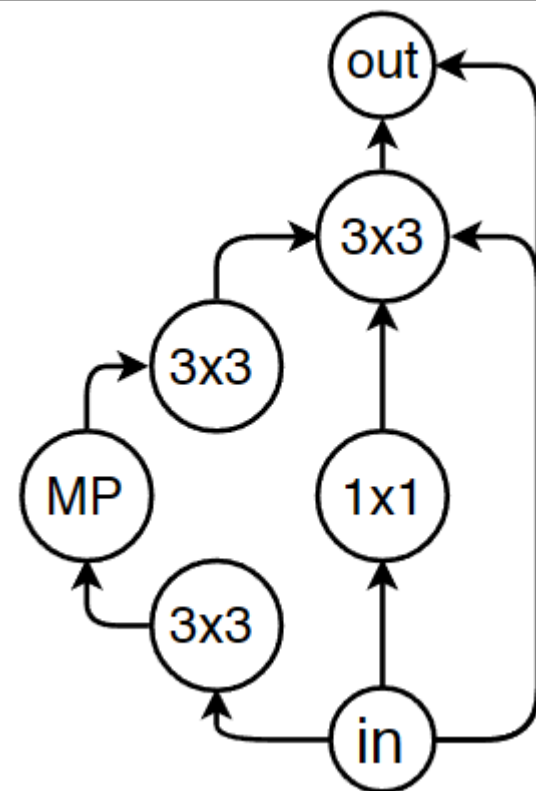
$$G = (V, E, \mathbf{o})$$

Onde:

$$V = \{v_{in}, v_1, v_2, v_3, v_4, v_5, v_{out}\}$$

$$E \subseteq V \times V$$

$$\mathbf{o} = [o_1, o_2, o_3, o_4, o_5]$$



Modelo de Sistema

Cada nó intermediário escolhe uma operação em:

$$\mathcal{O} = \{\text{conv } 3 \times 3, \text{conv } 1 \times 1, \text{max-pool } 3 \times 3\}$$

Restrições do espaço:

$$L = |\mathcal{O}| = 3$$

$$L = 3$$

$$V \leq 7$$

$$V = \{v_{in}, v_1, v_2, v_3, v_4, v_5, v_{out}\}$$

$$|E| \leq 9$$

Todas as convoluções usam:

Convolution $\rightarrow$ Batch Normalization $\rightarrow$ ReLU

L é o número de **labels**, ou seja, o número de operações possíveis que cada nó intermediário pode receber.

E é o conjunto de **arestas**, ou conexões direcionadas, entre os nós da célula.

Modelo de Sistema

- Os autores precisam de uma forma computacional de representar cada arquitetura. Para isso, usam uma matriz de adjacência 7×7 e uma lista de operações.
- A matriz indica quais conexões existem entre os nós. Como o grafo é direcionado e acíclico, a matriz é triangular superior. Isso força as conexões a irem de nós anteriores para nós posteriores, evitando ciclos.
- A lista $\mathbf{o}$ armazena as operações dos cinco nós intermediários. O nó de entrada e o nó de saída são fixos, então eles não recebem operações dentro dessa lista.

$$M \in \{0, 1\}^{7 \times 7}$$

$$\mathbf{o} = [o_1, o_2, o_3, o_4, o_5]$$

Onde:

- M : matriz de adjacência
- $M_{ij} = 1$: existe aresta de v_i para v_j
- $M_{ij} = 0$: não existe aresta
- matriz triangular superior: impede ciclos
- $\mathbf{o}$: lista de operações dos cinco nós intermediários

Essa codificação é geral e simples. Porém, ela tem um problema: diferentes matrizes podem representar arquiteturas computacionalmente equivalentes. Por isso, o benchmark precisa remover duplicatas por isomorfismo de grafos.

Modelo de Sistema

Tamanho bruto do search space

Com 7 nós, existem 21 possíveis arestas na matriz triangular superior.

Cada aresta pode estar presente ou ausente:

$$2^{21}$$

Cada um dos 5 nós intermediários pode receber 1 entre 3 operações:

$$3^5$$

Logo:

$$|\mathcal{A}_{bruto}| = 2^{21} \cdot 3^5 \approx 510 \text{ milhões}$$

Modelo de Sistema

Nem toda arquitetura codificada é válida.

Arquiteturas removidas:

- grafos sem caminho da entrada até a saída
- grafos com mais de 9 arestas
- grafos que não produzem uma rede executável
- grafos diferentes na codificação, mas equivalentes por isomorfismo

$$|\mathcal{A}_{NB101}| \approx 423\,000$$

Modelo de Sistema

Todas as arquiteturas são treinadas com o mesmo protocolo.

Configuração principal:

- dataset: CIFAR-10
- treino: 40 mil imagens
- validação: 10 mil imagens
- teste: 10 mil imagens
- otimizador: RMSProp
- loss: cross-entropy
- regularização: L2 weight decay
- learning rate com cosine decay
- hardware: TPU v2
- implementação: TensorFlow

TPU v2

- Acelerador de machine learning desenvolvido pelo Google.
- Hardware de data center, não uma GPU comum de PC.
- **Performance por chip: aproximadamente 45 TFLOPS.**
- **Configuração TPU v2-8: 8 núcleos, aproximadamente 180 TFLOPS.**
- **Memória TPU v2-8: 64 GB de HBM.**

Modelo de Sistema

Cada arquitetura é treinada com quatro budgets:

$$E_{stop} \in \{4, 12, 36, 108\}$$

Forma apresentada pelos autores:

$$E_{stop} \in \left\{ \frac{E_{max}}{3^3}, \frac{E_{max}}{3^2}, \frac{E_{max}}{3}, E_{max} \right\}$$

com:

$$E_{max} = 108$$

Cada arquitetura é treinada 3 vezes, com inicializações diferentes.

***O resultado final é grande: cerca de 423 mil arquiteturas, 3 repetições e 4 budgets.
Isso produz mais de 5 milhões de modelos treinados.***

Modelo de Sistema

O benchmark armazena uma função de consulta:

$$\mathcal{D}_{NB101}(A, E, t) \rightarrow \{Acc_{train}, Acc_{val}, Acc_{test}, T_{train}, P\}$$

Onde:

- A : arquitetura consultada
- E : budget de épocas
- t : repetição independente do treino
- Acc_{train} : acurácia de treino
- Acc_{val} : acurácia de validação
- Acc_{test} : acurácia de teste
- T_{train} : tempo de treinamento
- P : número de parâmetros treináveis

Modelo de Sistema

Cada consulta possui um custo de tempo:

$$T_{train}(A_i, E_i)$$

O tempo acumulado da busca é:

$$T_{acc}(i) = \sum_{j=1}^i T_{train}(A_j, E_j)$$

A busca para quando:

$$T_{acc}(i) > T_{limite}$$

Assim, algoritmos são comparados sob uma restrição de tempo equivalente.

Modelo de Sistema

Depois da busca, avalia-se a melhor arquitetura encontrada:

$$\hat{A}_i$$

A métrica usada é o test regret:

$$r(\hat{A}_i) = f(\hat{A}_i) - f(A^*)$$

Onde:

- $f(\hat{A}_i)$: acurácia média de teste da melhor arquitetura encontrada
- A^* : melhor arquitetura de todo o dataset
- $f(A^*)$: melhor acurácia média de teste disponível
- $r(\hat{A}_i)$: distância entre o algoritmo e o ótimo global do benchmark

Modelo de Sistema

Um único experimento de NAS pode ser enganoso.

O protocolo recomenda executar muitas buscas independentes:

$$\text{rollout}_1, \text{rollout}_2, \dots, \text{rollout}_R$$

Motivos:

- algoritmos de busca são estocásticos
- treinamento neural também é estocástico
- uma execução isolada pode favorecer um método por sorte
- a distribuição dos resultados mostra robustez
- curvas de regret ao longo do tempo mostram velocidade de busca

Os autores defendem que os algoritmos sejam avaliados com muitas execuções independentes. No paper, eles usam 500 trials independentes para comparar algoritmos como random search, regularized evolution, SMAC, TPE, Hyperband, BOHB e reinforcement learning.

Resultados

Resultado principal:

- maioria das arquiteturas converge
- muitas chegam a quase 100% de acurácia de treino
- maioria dos modelos supera 90% em validação e teste
- melhor arquitetura alcança:

$$Acc_{test}^{mean} = 94.32\%$$

Comparações internas:

- célula tipo ResNet: 93.12%
- célula tipo Inception: 92.95%
- ResNet-56 original: 93.03%

Resultados

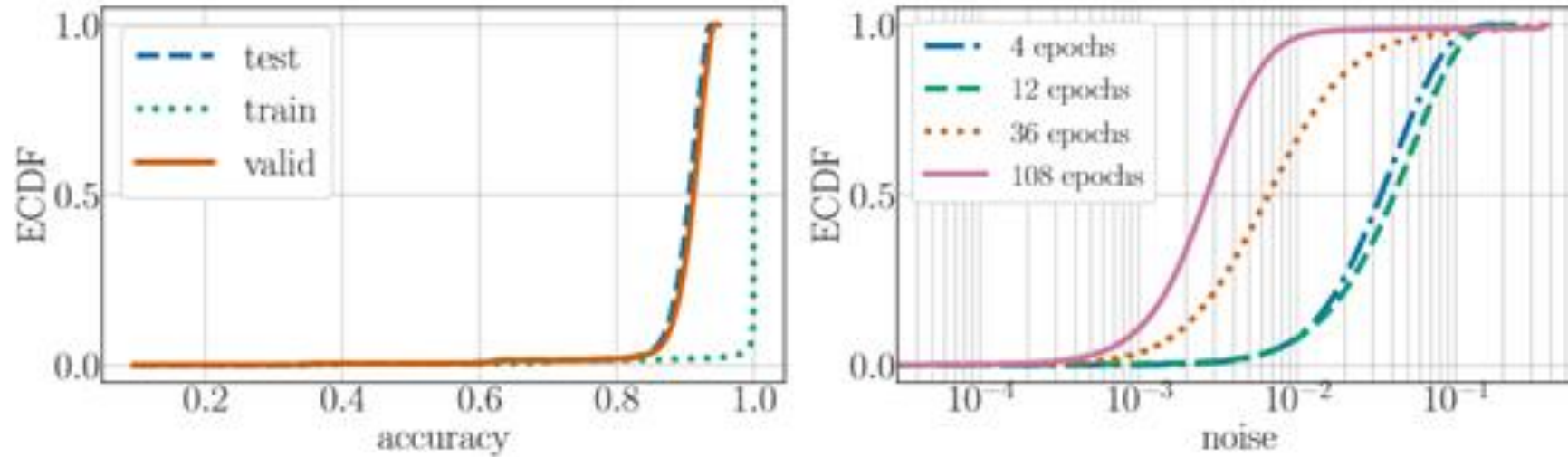


Figure 2: The empirical cumulative distribution (ECDF) of all valid configurations for: (left) the train/valid/test accuracy after training for 108 epochs and (right) the noise, defined as the standard deviation of the test accuracy between the three trials, after training for 12, 36 and 108 epochs.

Resultados

O paper investiga se arquiteturas parecidas também têm desempenhos parecidos.

A proximidade entre duas arquiteturas é medida por **edit distance**:

$$d(A_i, A_j)$$

onde $d(A_i, A_j)$ é o menor número de alterações necessárias para transformar a arquitetura A_i na arquitetura A_j .

Cada alteração pode ser:

- trocar a operação de um nó
- adicionar uma aresta
- remover uma aresta

Resultado principal:

- arquiteturas com baixa distância tendem a ter acurácias parecidas
- a correlação entre desempenho e proximidade desaparece após distância próxima de 6
- as melhores arquiteturas possuem uma vizinhança local de arquiteturas também boas
- isso indica que o search space não é aleatório, ele possui estrutura explorável

Resultados

As melhores arquiteturas são muito raras no espaço bruto.

O espaço codificado possui aproximadamente:

510 milhões de pontos

Entre eles, apenas:

11 570

correspondem às arquiteturas estatisticamente próximas da melhor arquitetura.

Logo, a chance de encontrar uma arquitetura top por amostragem aleatória é aproximadamente:

$$P(\text{top}) = \frac{11\,570}{510\,000\,000} \approx 2.27 \times 10^{-5} \approx \frac{1}{50\,000}$$

Ou seja:

Random Search puro dificilmente acerta diretamente uma arquitetura top.

Resultados

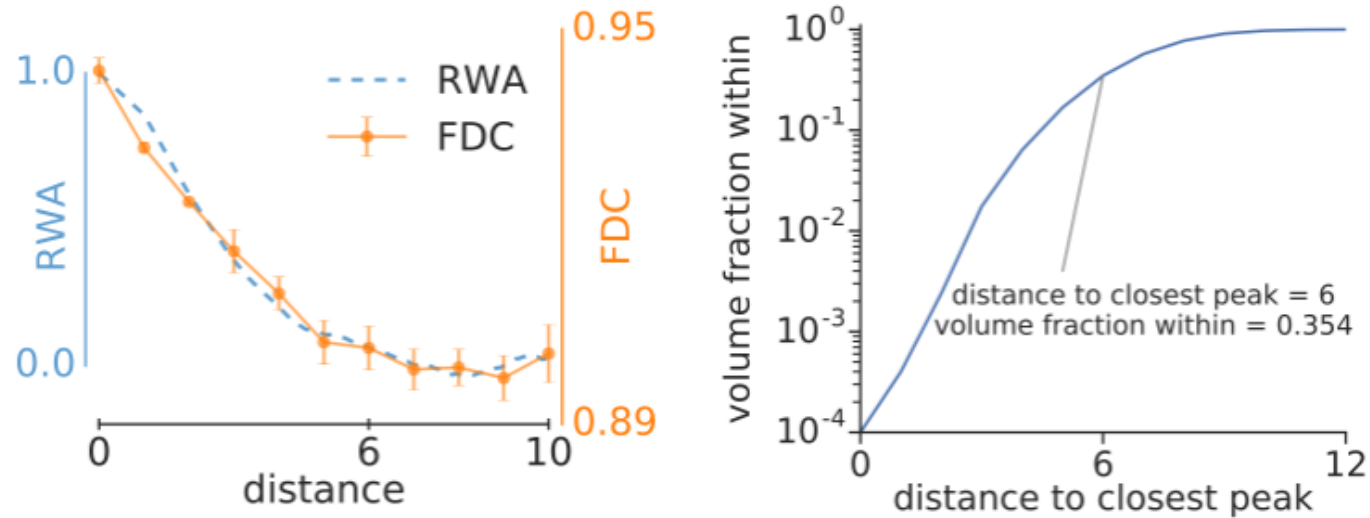


Figure 6: (left) RWA for the full space and the FDC relative to the global maximum. To plot both curves on a common horizontal axis, the autocorrelation curve is drawn as a function of the square root of the autocorrelation shift, to account for the fact that a random walk reaches a mean distance $\sqrt{N}$ after N steps. (right) Fraction of the search space volume that lies within a given distance to the closest high peak.

RWA, Random-Walk Autocorrelation, mede a autocorrelação das acurácias observadas durante uma caminhada aleatória pelo search space.

Se arquiteturas próximas têm acurácias parecidas, o RWA fica alto para pequenas distâncias.

FDC, Fitness-Distance Correlation, mede a relação entre a distância de uma arquitetura até a melhor arquitetura global e seu desempenho.

Se arquiteturas próximas do ótimo também possuem alta acurácia, existe uma bacia local ao redor do máximo.

Resultados

Algoritmos avaliados:

- Random Search, RS
- Regularized Evolution, RE
- SMAC
- TPE
- Hyperband, HB
- BOHB
- Reinforcement Learning, RL

Protocolo:

- 500 execuções independentes
- eixo x: tempo estimado de wall-clock
- eixo y: test regret
- menor regret indica arquitetura mais próxima do ótimo global

$$r(\hat{A}_i) = f(\hat{A}_i) - f(A^*)$$

- $f(\hat{A}_i)$: acurácia média de teste da melhor arquitetura encontrada
- A^* : melhor arquitetura de todo o dataset
- $f(A^*)$: melhor acurácia média de teste disponível
- $r(\hat{A}_i)$: distância entre o algoritmo e o ótimo global do benchmark

Algoritmos Testados

O **RS, Random Search**, é o método mais simples. Ele apenas sorteia arquiteturas no espaço de busca e consulta o desempenho de cada uma. Ele não aprende com tentativas anteriores. Por isso, funciona como baseline: qualquer método mais complexo precisa justificar seu ganho em relação a essa busca aleatória.

O **RE, Regularized Evolution**, usa uma população de arquiteturas. A cada iteração, o método seleciona algumas candidatas, escolhe uma arquitetura com bom desempenho, aplica uma mutação e adiciona o novo indivíduo à população. A regularização vem da remoção das arquiteturas mais antigas, o que força renovação e reduz dependência de soluções antigas.

O **SMAC, Sequential Model-based Algorithm Configuration**, é uma forma de Bayesian optimization. Ele constrói um modelo substituto, normalmente baseado em random forest, para estimar o desempenho de arquiteturas ainda não avaliadas. Em vez de testar candidatos totalmente às cegas, ele usa essa estimativa para escolher arquiteturas mais promissoras.

Algoritmos Testados

O **TPE, Tree-structured Parzen Estimator**, também é Bayesian optimization, mas modela o problema por densidades. Ele divide as arquiteturas já avaliadas em dois grupos, boas e ruins, aprende distribuições para cada grupo e tenta amostrar novas arquiteturas que sejam mais prováveis no grupo bom do que no grupo ruim.

O **HB, Hyperband**, é um método multi-fidelity. Ele testa muitas arquiteturas com poucos recursos, por exemplo poucas épocas de treinamento, e mantém apenas as melhores para avaliações mais longas. A lógica é economizar custo eliminando cedo candidatos pouco promissores.

O **BOHB, Bayesian Optimization and Hyperband**, combina as duas ideias. Ele usa a alocação progressiva de recursos do Hyperband, mas escolhe novos candidatos usando um modelo probabilístico. Assim, não depende apenas de amostragem aleatória nos estágios iniciais.

O **RL, Reinforcement Learning**, trata a geração de arquiteturas como uma política aprendível. O método gera uma arquitetura, recebe uma recompensa baseada no desempenho e ajusta a política para aumentar a probabilidade de escolhas que produziram bons resultados. No NAS-Bench-101, essa política é implementada de forma simples com distribuições categóricas otimizadas por reinforce.

RE - Regularized Evolution

População inicial:

$$P = \{A_1, A_2, A_3, A_4, A_5\}$$

Cada arquitetura já tem uma acurácia de validação:

$$A_1 = 91.0\%, \quad A_2 = 92.4\%, \quad A_3 = 90.8\%, \quad A_4 = 93.1\%, \quad A_5 = 91.7\%$$

Passos:

1. Sorteia um subconjunto da população:

$$\{A_2, A_4, A_5\}$$

2. Escolhe a melhor do subconjunto:

$$A_4 = 93.1\%$$

3. Aplica uma mutação em A_4 :

$$A_4 \rightarrow A_6$$

Exemplos de mutação:

- trocar uma operação
 - adicionar uma aresta
 - remover uma aresta
4. Avalia A_6 e adiciona à população.
 5. Remove a arquitetura mais antiga, não necessariamente a pior.

SMAC - Sequential Model-based Algorithm Configuration

Arquiteturas já avaliadas:

$$A_1 = 91.0\%, \quad A_2 = 92.5\%, \quad A_3 = 90.6\%, \quad A_4 = 93.0\%$$

O SMAC treina um modelo substituto:

$$\hat{f}(A) \approx Acc_{val}(A)$$

No SMAC, esse modelo costuma ser uma **random forest**.

Depois, ele estima o desempenho de novas arquiteturas:

$$\hat{f}(A_5) = 92.1\%$$

$$\hat{f}(A_6) = 93.4\%$$

$$\hat{f}(A_7) = 91.8\%$$

TPE - Tree-structured Parzen Estimator

Arquiteturas avaliadas:

$$A_1 = 91.0\%, \quad A_2 = 93.2\%, \quad A_3 = 90.5\%, \quad A_4 = 92.8\%, \quad A_5 = 89.9\%$$

Define-se um limiar, por exemplo top 40%.

Grupo bom:

$$\mathcal{G}_{bom} = \{A_2, A_4\}$$

Grupo ruim:

$$\mathcal{G}_{ruim} = \{A_1, A_3, A_5\}$$

O TPE estima duas densidades:

$$l(A) = p(A \mid A \text{ é bom})$$

$$g(A) = p(A \mid A \text{ é ruim})$$

HB - Hyperband

No NAS-Bench-101, os budgets são:

$$E \in \{4, 12, 36, 108\}$$

Exemplo simples:

Começa com muitas arquiteturas e pouco treino:

$$9 \text{ arquiteturas} \times 4 \text{ épocas}$$

Mantém as melhores:

$$3 \text{ arquiteturas} \times 12 \text{ épocas}$$

Mantém as melhores novamente:

$$1 \text{ arquitetura} \times 36 \text{ ou } 108 \text{ épocas}$$

Fluxo:

muitos candidatos, baixo custo $\rightarrow$ poucos candidatos, alto custo

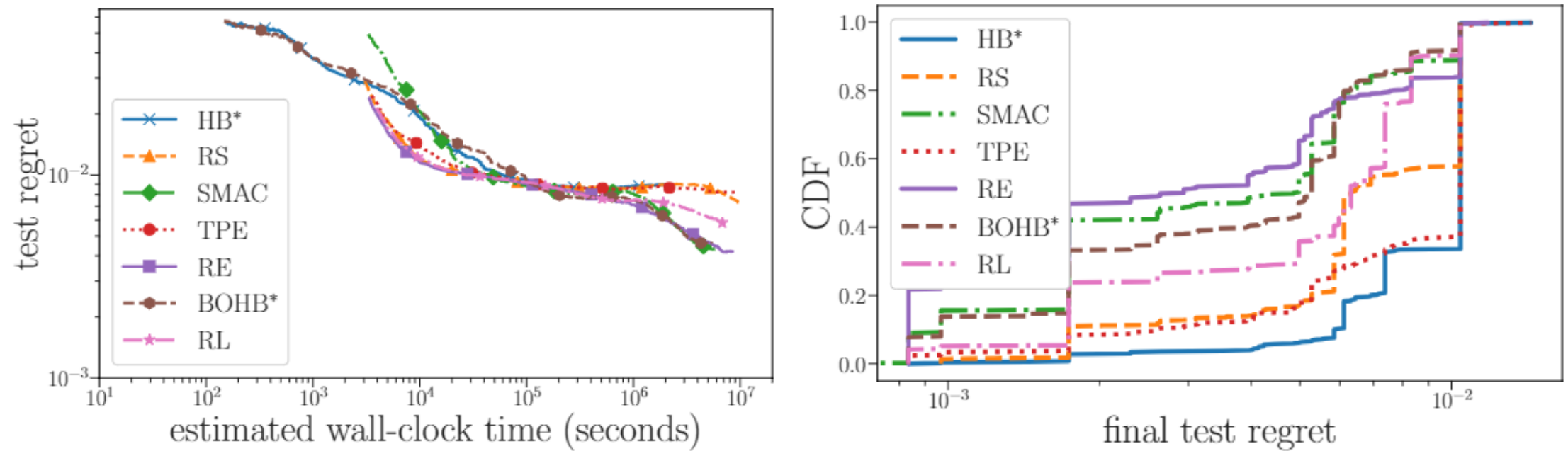


Figure 7: (left) Comparison of the performance of various search algorithms. The plot shows the mean performance of 500 independent runs as a function of the estimated training time. (right) Robustness of different optimization methods with respect to the seed for the random number generator. *HB and BO-HB are budget-aware algorithms which query the dataset a shorter epoch lengths. The remaining methods only query the dataset at the longest length (108 epochs).

- Random Search é uma baseline forte no início
- RE, BOHB e SMAC vencem no médio e longo prazo
- TPE tem dificuldade no espaço discreto do NAS-Bench-101
- Hyperband sofre quando poucos epochs não predizem bem 108 epochs
- RL melhora sobre Random Search, mas converge lentamente
- robustez exige muitas execuções independentes

**RE, BOHB e SMAC
são os métodos
mais fortes no
benchmark**

Conclusões

O NAS-Bench-101 propõe um benchmark tabular para pesquisa em Neural Architecture Search.

A contribuição central não é uma nova arquitetura, mas uma infraestrutura experimental para NAS.

O benchmark permite:

- avaliar algoritmos de busca sem treinar redes do zero
- comparar métodos sob o mesmo search space
- reduzir o custo de reprodução de experimentos
- analisar propriedades globais do espaço de arquiteturas
- medir robustez com muitas execuções independentes

Mensagem central:

NAS-Bench-101 torna a avaliação de NAS mais rápida, acessível e reprodutível.

Trabalhos Futuros

Os autores posicionam o NAS-Bench-101 como o primeiro passo de uma sequência de benchmarks mais rigorosos para NAS.

Direções naturais:

- criar benchmarks maiores
- incluir search spaces mais modernos
- avaliar datasets além de CIFAR-10
- expandir para tarefas além de classificação
- incluir mais métricas de custo
- melhorar a integração entre NAS e HPO
- comparar mais famílias de algoritmos

Limitações

Por que os modelos não atingem estado da arte?

O paper aponta três causas principais:

1. Search space restrito

Não inclui arquiteturas mais complexas usadas por métodos de NAS de maior escala.

2. Sem augmentation trick caro

Não aumenta fortemente profundidade, largura e tempo de treino.

3. Sem regularizações avançadas

Não usa técnicas como Cutout, ScheduledDropPath ou decoupled weight decay.

O benchmark é controlado, mas possui escopo restrito.

Principais limitações:

- usa apenas CIFAR-10
- avalia apenas arquiteturas CNN baseadas em células
- search space possui poucas operações candidatas
- macroestrutura da rede é fixa
- hiperparâmetros de treinamento são fixos
- não representa arquiteturas modernas mais complexas
- não atinge estado da arte absoluto em CIFAR-10

Perguntas?

Quiz

<https://forms.gle/7uN4r39chu6JFJrM7>